



Modular Computer Vision System

Prepared for

Dr/ Mahmoud Gamal

Professor, Faculty of Computing And

Data science Alexandria university

By

Name	ID
Mesk Khaled	23011537
Shahd Fawzy	23012087
Ahmed Mohamed Shaaban	23011213
Moaaz Salama	23012085

May 2026

Introduction

- This project develops a modular computer vision system for image processing and enhancement.
 - The system is designed in a Jupyter notebook using Python and common computer vision libraries.
 - The main objective is to create reusable processing steps that can be applied to different images and extended to future tasks.
 - The project focuses on combining classical image processing techniques such as color masking, sharpening, blurring, and blending.
 - The overall workflow is built to be simple, flexible, and easy to modify for additional computer vision experiments.
-

Task 1: Selective Object Enhancement

1. Task Description

- The first task identifies regions in an image that match a chosen target color to enhance a specific object while applying a different visual effect (blurring) to the rest of the image.
- The selected region is visually enhanced (sharpened and saturated) while the background is softened.
- The output is displayed as a triptych:
 - Original image
 - Color mask
 - Enhanced image

2. Method Used

- The image is converted from BGR to the HSV color space for more robust, lighting-independent color segmentation.
- Predefined HSV lower and upper bounds are used to isolate the target color and create a binary mask.
- Morphological operations (Open and Close) are applied to the mask to clean up small noise artifacts and fill in gaps.
- A strong Gaussian blur is applied to the mask to soften its edges, transitioning it into a normalized float array (0.0 to 1.0) for smooth alpha blending.
- The original image is processed to create a vibrant "foreground" layer by boosting its saturation and applying an unsharp mask to increase detail.
- A separate "background" layer is generated by heavily blurring the original image to reduce distraction.
- The final output is composited by linearly interpolating (alpha blending) between the enhanced foreground and blurred background using the soft mask.

3. Implemented Functions

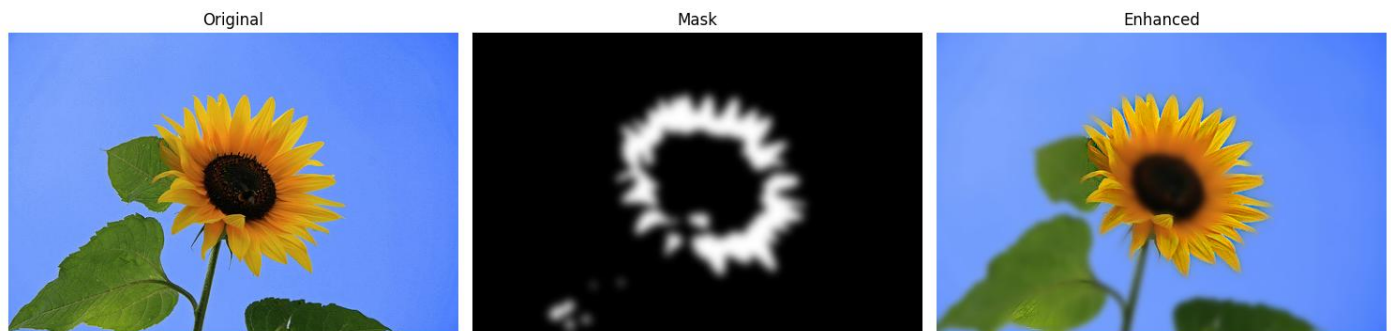
- **get_soft_mask()**
 - Creates a smooth, normalized alpha mask for the selected color.
 - Handles the HSV thresholding, morphological noise filtering, and edge-softening (Gaussian blur).
- **enhance_foreground()**

- A consolidated function that boosts the saturation of the target region to make the object stand out.
- Simultaneously sharpens the image using an unsharp masking technique.
- **selective_object_enhancement()**
 - The main orchestrator function. It loads the image, generates the mask and the two distinct visual layers, and performs the final mathematical alpha blending.
- **show_results()**
 - Handles the visualization using Matplotlib, displaying the original image, the computed 2D soft mask, and the final result side-by-side.

4. Results

- The system successfully highlights the selected object based on its color.
- The background becomes less dominant due to the contrasting blurring effect.
- The final output looks highly focused, drawing the viewer's eye to the vibrant target object.
- The intermediate mask visualization allows users to verify that the color detection step is working accurately.

Enhanced yellow in sunflower.jpg



Task 2: Comparative Study of Sharpening Pipelines for X-Ray Images

1. Task Description

- The goal of this task is to design and evaluate three distinct image enhancement strategies (pipelines) to improve the clarity of X-ray images. X-rays often suffer from low contrast and soft edges, making fine anatomical structures difficult to see. These pipelines aim to accentuate edges and improve local visibility for better diagnostic analysis

2. Implementation Methodology

- In this task, we implemented the pipelines using OpenCV built-in primitives (such as GaussianBlur, Laplacian, and Sobel) as mathematical building blocks. While these low-level functions handle the heavy computation, the overall execution logic and the way these steps are combined remain a custom design.

3. Detailed Pipeline Analysis

- **Pipeline 1: Unsharp Masking (Frequency Separation)**
 - **Logic:** It isolates the high-frequency components (edges) by subtracting a blurred version of the image from the original. This "mask" is then amplified and added back to the original image.

- **Goal:** To provide balanced sharpening without overly distorting the original pixel distribution.
- **Pipeline 2: Laplacian Enhancement (High-Pass Injection)**
 - **Logic:** This pipeline first normalizes local contrast using **CLAHE**. It then calculates the **Laplacian** (second-order derivative) to find regions of rapid intensity change. These sharp edges are then "injected" back into the balanced image.
 - **Goal:** Specifically targeted at enhancing very fine line structures, such as micro-fractures.
- **Pipeline 3: Gradient Injection (Sobel + Adaptive Equalization)**
 - **Logic:** It uses **Sobel operators** to calculate directional gradients (edges). These gradients are merged with a heavily equalized (CLAHE) base using weighted blending.
 - **Goal:** To maximize the visibility of soft tissue and lung vascularity by forcing high contrast and strong edge definition simultaneously.

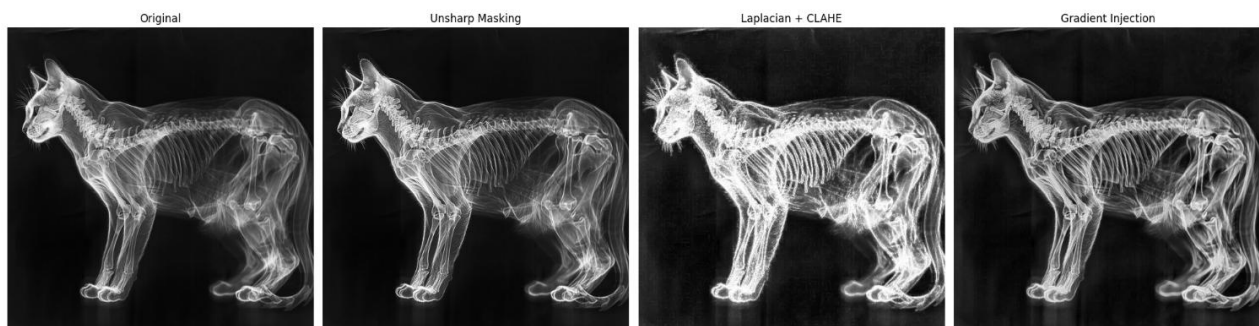
4. Quantitative Evaluation

- The performance is measured using standard metrics implemented in our library:
 - **PSNR (Peak Signal-to-Noise Ratio):** Measures the fidelity of the enhanced image compared to the original.
 - **SSIM (Structural Similarity):** Ensures that the enhancement has not corrupted the actual anatomical structures.
 - **Edge Strength Gain:** Tracks how much the edge gradients have been amplified.

5. Visual Results and Comparison

- As seen in the generated comparison figure, while Pipeline 1 offers the most natural look, **Pipeline 3 (Gradient Injection)** often provides the highest level of detail for soft tissues, and **Pipeline 2 (Laplacian)** excels in highlighting dense bone structures.

X-Ray Enhancement Comparison
Source: cat skeleton xray.jpg



Task 3: Intelligent Auto Image Enhancement System

1. Objective

- The objective of this task was to design and implement an intelligent, context-aware image enhancement pipeline. Rather than applying a single, static set of filters to every image, the system autonomously analyzes the statistical properties of each input image and dynamically routes it through the most appropriate enhancement strategy to correct specific exposure, contrast, or color issues.

2. Statistical Analysis & Methodology

- To determine the optimal enhancement path, the system first converts the input image to grayscale to evaluate its baseline luminance characteristics. Four key metrics are extracted:
 - **Mean (Brightness):** Determines the average overall lightness of the image.
 - **Standard Deviation (Contrast):** Measures the spread of pixel intensities, indicating whether the image is flat/washed out or has healthy contrast.
 - **Under-clip Ratio:** Calculates the percentage of pixels crushed into the lowest histogram bins (near pure black), indicating underexposure.
 - **Over-clip Ratio:** Calculates the percentage of pixels pushed into the highest histogram bins (near pure white), indicating overexposure.

3. Dynamic Routing & Strategies

- Based on the extracted statistics, the system categorizes the image and applies a targeted combination of algorithms:
 - **Underexposed (Mean < 90 or Under-clip > 1%):**
 - If the image also suffers from low contrast (Std Dev < 45), it receives Gamma Correction (gamma = 0.95) to lift midtones, followed by CLAHE to restore local contrast.
 - If the contrast is already acceptable, only Gamma Correction is applied to brighten the scene.
 - **Overexposed (Mean > 170 or Over-clip > 1%):**
 - The image receives Gamma Correction (gamma = 1.2) to pull down harsh highlights, followed by Auto-Contrast Stretching (clipping the extreme 1.5% and 98% percentiles) to redistribute the remaining pixel values across the visible spectrum.
 - **Low Contrast (Standard Deviation < 50):**
 - If the image is properly exposed but flat, the system applies CLAHE (Contrast-Limited Adaptive Histogram Equalization) to enhance local details without overblowing global brightness.
 - **Normal / Well-Exposed:**
 - Images with healthy brightness and contrast bypass aggressive manipulation and instead receive an Auto White-Balance correction to neutralize any unwanted color casts.

4. Algorithm Implementation Details

- To preserve image quality and prevent color distortion, several technical best practices were implemented in the algorithms:
 - **LAB Color Space Operations:** Both Gamma Correction and CLAHE are applied exclusively to the 'L' (Luminance) channel of the LAB color space. This strictly limits modifications to brightness and contrast, ensuring the original color hues and saturations are not accidentally shifted or degraded.
 - **Gray-World White Balance:** The normal image strategy utilizes a gray-world assumption, calculating the mean of the BGR channels and scaling them toward a neutral gray midpoint (128.0) to remove color tints.

- Universal Sharpening (Unsharp Masking): Regardless of the chosen strategy, every image passes through a final Unsharp Masking filter. This subtracts a slightly blurred (Gaussian) version of the image from the original to isolate high-frequency details, which are then added back to crisp up the final output.

5. Execution and Results

- The pipeline was packaged into a modular processing loop that reads images, evaluates them, routes them, and applies the universal sharpening pass. During execution, the system processes a batch of test images (rose.jpg, sunflower.jpg, and cat skeleton xray.jpg), autonomously determining the optimal mathematical approach for each unique lighting condition, and outputs a side-by-side visual comparison alongside the calculated statistics.

Auto Enhancement | Strategy: underexposed | mean=46.6 std=36.0 under_clip=0.001 over_clip=0.002
Original Enhanced



Task 4: Document Cleaning System

1. Objective

- The objective of this task was to develop and evaluate classical computer vision techniques for restoring degraded document images. Scanned documents often suffer from uneven illumination, heavy shadows, and sensor noise, making them difficult to read or process via Optical Character Recognition (OCR). The goal was to test three independent methods to isolate the textual foreground and produce a clean, binarized output.

2. Methodology & Evaluated Techniques

- Rather than building a single rigid pipeline, the system was designed to independently evaluate three distinct mathematical approaches to document binarization. Each method was applied directly to the original noisy grayscale image to isolate its specific effects:
 - **Method A: Median Background Subtraction**
 - Approach: This method estimates the uneven lighting by applying a severe median blur (kernel size = 23) to the image, which mathematically eliminates the thin, sharp text while preserving the broad gradients of the background shadows.
 - Execution: The estimated background is subtracted from the original image. The contrast is then normalized to stretch the values across the full 0-255 spectrum, and the image is inverted to produce standard dark text on a white background.

- **Method B: Edge Dilation Method**

- Approach: This structural approach ignores lighting entirely and focuses on high-frequency changes (edges).
- Execution: Canny Edge Detection (thresholds 50 and 150) is used to locate the outlines of the characters. Because edge detection only finds the borders of the text, a morphological Dilation operation (using a 3x3 kernel) is applied to thicken these edges into solid structural characters. The image is then inverted.

- **Method C: Gaussian Adaptive Thresholding**

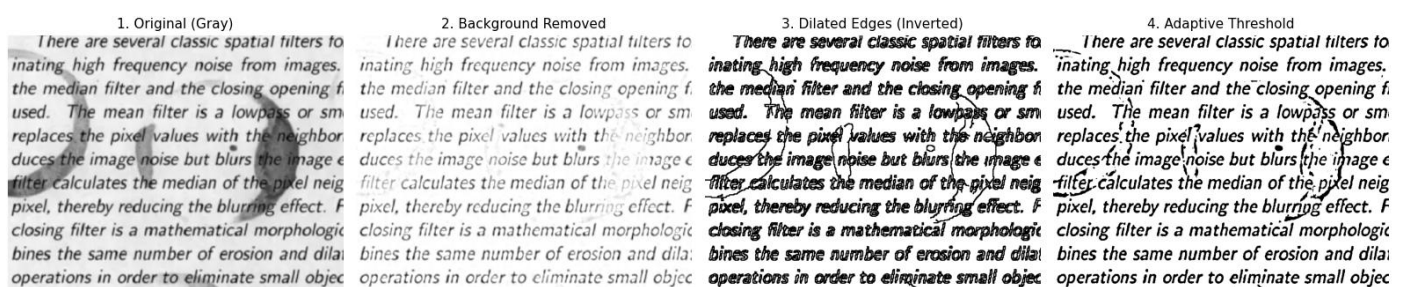
- Approach: Standard global thresholding fails on unevenly lit documents because a single threshold value will turn shadowed areas entirely black.
- Execution: To counter this, cv2.adaptiveThreshold is utilized with a block size of 35. Instead of a global value, this algorithm calculates a unique threshold for every pixel based on the Gaussian-weighted average of its local 35x35 neighborhood, effectively neutralizing large-scale shadows.

3. Implementation Details

- **Libraries:** The system relies on OpenCV (cv2) for all core matrix mathematics, filtering, and morphological operations.
- **Color Space:** All processing is done strictly in single-channel Grayscale, as color data is largely irrelevant for textual document structures and adds unnecessary computational overhead.
- **Visualization:** Matplotlib is utilized to generate a side-by-side 1x4 subplot, allowing for direct visual comparison between the original dirty document and the output of the three techniques.

4. Results and Analysis

- Testing the system on the sample degraded image ("dirty document.png") revealed distinct advantages for each technique:
 - Median Subtraction proved highly effective at neutralizing broad shadows while maintaining the original font weight and thickness, though it can sometimes leave behind small localized noise.
 - Edge Dilation excelled at completely ignoring all background gradients and noise, producing an extremely high-contrast result, though the character structures can sometimes appear artificially thick or slightly distorted depending on the dilation kernel.
 - Adaptive Thresholding generally produced the crispest, most uniform "document-like" result. By dynamically adjusting to local lighting conditions, it successfully binarized the text in both the brightly lit and heavily shadowed regions without losing character detail.



Task 5: Panorama Image Stitching

1. Task Description

- Panorama image stitching is a fundamental computer vision technique that combines multiple overlapping images of the same scene into a single, seamless wide-angle image. This technology is widely used in photography, virtual reality, surveillance, and mapping applications.
- This project implements a modular, from-scratch panorama stitching system that takes multiple overlapping images and produces a high-quality panoramic output using feature-based alignment, geometric transformations, and advanced blending techniques.

2. Objectives

- Detect and match distinctive features across overlapping images.
- Estimate geometric transformations (**homography**) using robust algorithms.
- Warp and align images into a common coordinate system.
- Blend overlapping regions seamlessly without visible artifacts.
- Design a modular, reusable library architecture.
- Handle real-world challenges (different image sizes, lighting variations).

3. Detailed Pipeline Analysis

Step 1: Feature Detection and Description

- Key points are detected using SIFT.
- Descriptors are extracted to represent local image features.
- This allows matching between images even under scale and rotation changes.

Step 2: Feature Matching

- A FLANN-based matcher is used to find correspondences between descriptors.
- Lowe's ratio test is applied to filter out incorrect matches.

Step 3: Homography Estimation

- RANSAC is used to compute a robust transformation matrix.
- Outliers are removed to improve alignment accuracy.

Step 4: Image Warping

- One image is warped into the coordinate space of the other using perspective transformation.
- A larger canvas is created to fit both images.

Step 5: Blending

- Non-overlapping regions are copied directly.
- Overlapping regions are blended using a smooth Gaussian-based approach.
- This reduces visible seams between images.

Step 6: Cropping

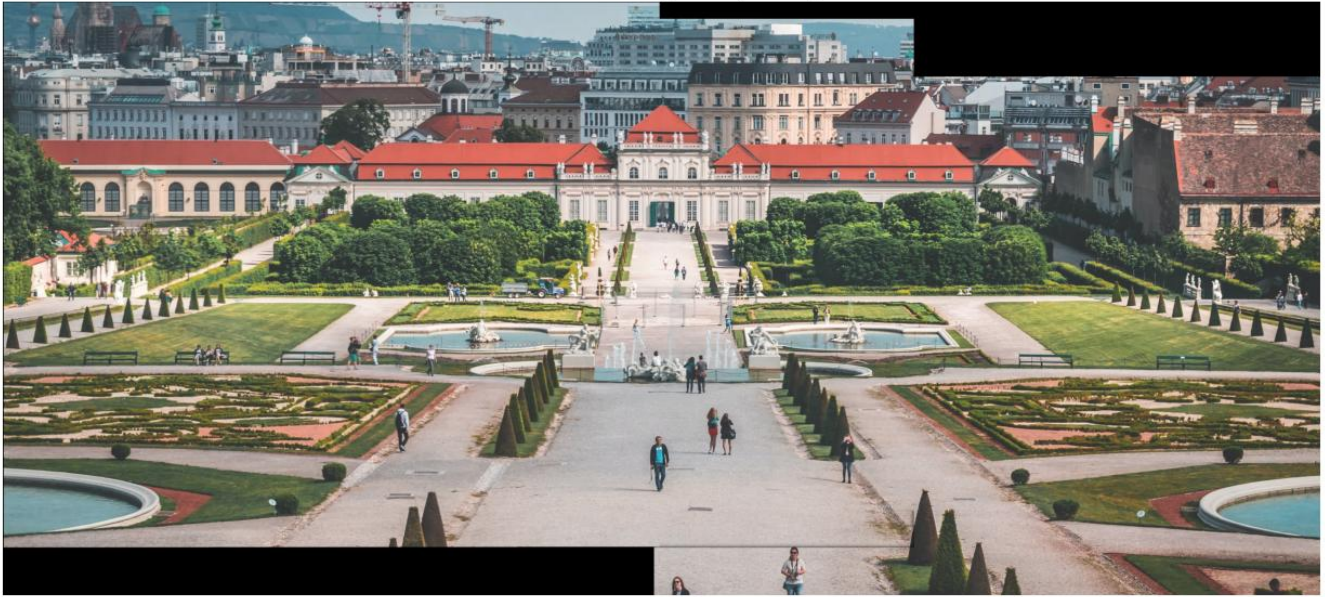
- The final panorama is cropped to remove black borders.

4. Results

- The system successfully aligns and merges overlapping images.
- The final panorama shows a wider view of the scene.
- Blending reduces visible seams between images.

- Feature matching visualization helps verify alignment quality.

Final Panorama: img1 + img2 + img3



Task 6: Transformed Object Recognition

1. Objective

- Design and implement a robust, modular system capable of detecting and localizing a reference object in query images despite geometric transformations, including rotation, translation, scaling, and mild perspective distortion. The solution must be implemented from scratch using core computer vision concepts and structured for reuse within the team's image processing library.

2. Design Strategy & Theoretical Background

- Instead of brute-force template matching (which fails under transformations), we adopted a feature-based recognition pipeline.
- We chose SIFT as the default detector/descriptor due to its proven scale and rotation invariance. FLANN-based k-NN matching ensures fast and accurate correspondence search. The pipeline is fully deterministic, requires no training, and runs entirely with OpenCV primitives.

Concept	Role in System
Keypoint Detection	Locates distinctive, repeatable points in the image (corners, blobs, edges)
Descriptor Extraction	Encodes local gradient/texture information around each keypoint into a numerical vector
Feature Matching	Compares descriptors between reference and query images to find correspondences
Lowe's Ratio Test	Filters ambiguous matches by comparing the nearest vs. second-nearest neighbor distances

Homography Estimation	Computes a 3×3 projective transformation matrix that maps reference points to query points
RANSAC	Robustly estimates homography while automatically rejecting outlier matches

3. Processing Pipeline

- The system follows a strict, modular 5-stage workflow:

i. Input Preparation

- Reference template and query image loaded (BGR format).
- Internal conversion to grayscale for descriptor computation.

ii. Feature Extraction

- **cv2.SIFT_create()** detects keypoints and computes 128-D descriptors.
- Handles scale-space pyramids internally → inherently scale/rotation invariant.

iii. Descriptor Matching & Filtering

- **knnMatch(k=2)** retrieves two best matches per descriptor.
- Lowe's ratio test (**distance < 0.75 × second_best_distance**) removes false correspondences.

iv. Geometric Verification

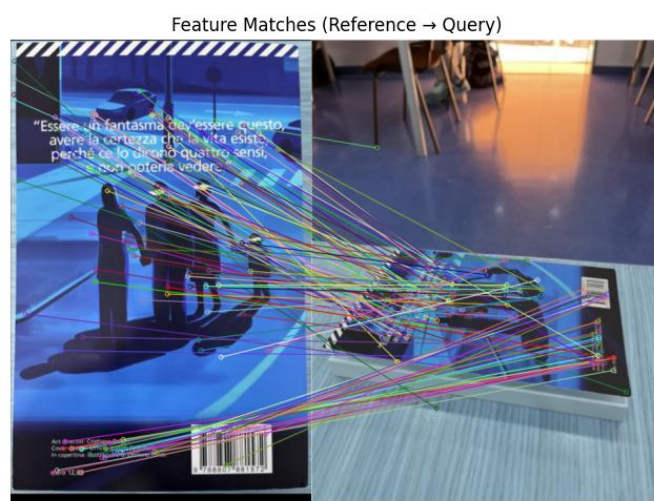
- Matched point pairs passed to **cv2.findHomography()** with RANSAC (threshold=5.0).
- Returns transformation matrix H and inlier mask.

v. Decision & Localization

- If **inliers ≥ min_threshold** (default: 10) → Object recognized.
- Reference corners transformed via **cv2.perspectiveTransform()** and drawn on query image.

4. Results

- The system outputs:
 - A boolean indicating whether the object is detected.
 - The query image with detected object boundary.
 - Number of inlier matches.
 - Visualization of feature correspondences.



Task 7: Depth Approximation from Two Images

1. Task Description

- This task estimates the relative depth of a scene by processing a stereo image pair — two photographs of the same scene taken from slightly different horizontal positions.
 - The pipeline extracts feature correspondences between the two images, rectifies them into a canonical stereo configuration, and computes a dense disparity map indicating how far each pixel has shifted between views.
 - Disparity is then converted into a depth map: pixels with high disparity are close to the camera, while pixels with low disparity are far away.
 - If the real distance between the two camera positions (the baseline) is provided, the output depth is expressed in centimetres; otherwise it is presented in normalised units.
 - The output is displayed as an eight-panel figure:
 - Original left image
 - Hole Map (showing invalid pixel locations and fill percentage)Right rectified image
 - Raw disparity map
 - Filled depth map (Magma colormap)
 - Grayscale Depth — Before Inversion (farther = brighter)
 - Grayscale Depth — After Inversion (closer = brighter)
 - Metric Depth — Before Inversion (Inferno colormap)
 - Metric Depth — After Inversion (Inferno colormap)

2. Method Used

- The image is converted to grayscale and contrast-enhanced using CLAHE (Contrast Limited Adaptive Histogram Equalisation) before feature detection, making the pipeline more robust to uneven lighting.
- SIFT (Scale-Invariant Feature Transform) is used to detect up to 8,000 keypoints per image and compute 128-dimensional descriptors for each.
- Brute-Force Matching with Lowe's ratio test (threshold 0.75) filters out ambiguous matches, and RANSAC is applied to the Fundamental Matrix estimation to retain only geometrically consistent inlier correspondences.
- The Essential Matrix is decomposed to recover the relative camera rotation and translation, which are used to perform stereo rectification. The alpha parameter is set to 0 to crop all black border regions from the rectified images.
- StereoSGBM (Semi-Global Block Matching) is used in place of the simpler StereoBM to compute the disparity map. SGM aggregates matching costs along multiple directions, producing smoother results with significantly fewer holes.
- A left-right consistency check is applied: the disparity is computed in both directions and pixels where the two estimates disagree by more than 2 pixels are marked as invalid, removing geometrically inconsistent matches.
- Hole filling is performed in three stages: morphological closing fills tiny gaps, Navier-Stokes inpainting recovers larger invalid regions, and a guided filter (or bilateral filter as fallback) smooths the result while preserving depth edges.

- The final depth map is computed as $\text{depth} = (\text{focal_length} \times \text{baseline}) / \text{disparity}$, and the values are clipped at the 1st and 99th percentiles to suppress outliers before colormap visualisation.

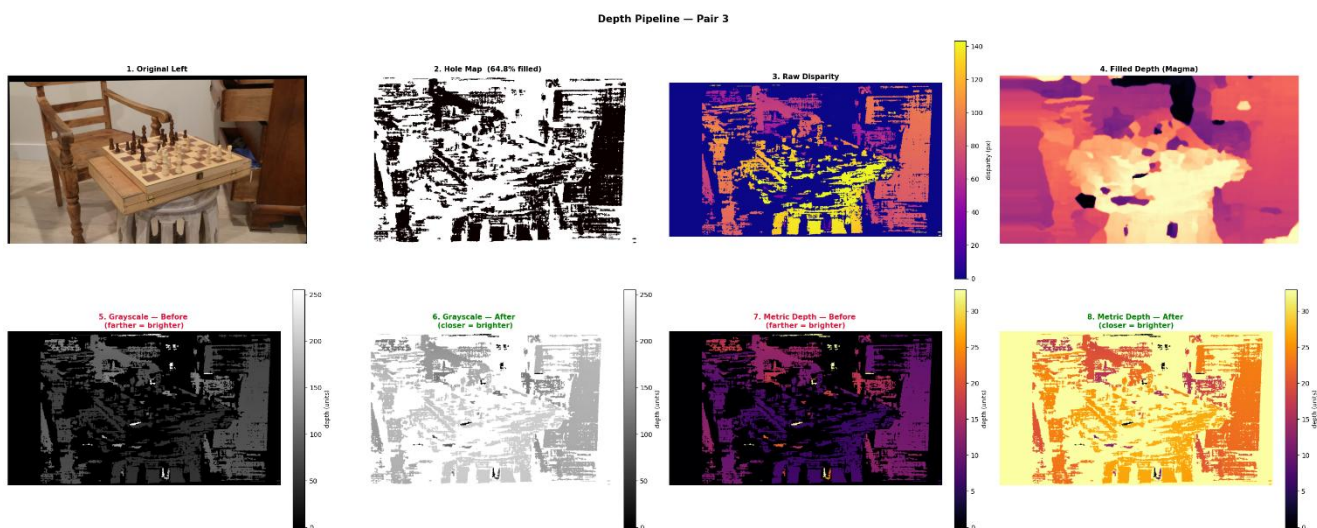
3. Implemented Functions

- **extract_focal_px():**
 - Reads focal length and 35mm-equivalent focal length from EXIF metadata embedded in the left image.
 - Derives the sensor crop factor and converts the physical focal length to pixels.
 - Falls back to a 70-degree horizontal field-of-view assumption if EXIF data is unavailable or out of range.
- **build_K():**
 - Constructs the 3×3 camera intrinsics matrix K from the extracted focal length and image centre coordinates.
 - Clamps the focal length to a valid range (0.5× to 3× the image width) to guard against corrupted EXIF values.
- **to_gray_clahe():**
 - Converts a colour image to grayscale and applies CLAHE preprocessing to improve local contrast before feature detection and disparity computation.
- **match_features():**
 - Runs SIFT on both images, applies Lowe's ratio test to the BFMatcher results, and estimates the Fundamental Matrix using RANSAC.
 - Returns the inlier point correspondences and the Fundamental Matrix for use in rectification.
- **rectify():**
 - Computes the Essential Matrix from the Fundamental Matrix and camera intrinsics, recovers the relative pose, and applies stereoRectify to align both images onto a common horizontal scanline plane.
 - Returns the rectified image pair, the rectified focal length, and the effective baseline.
- **compute_disparity():**
 - Runs StereoSGBM in both left-to-right and right-to-left directions to compute two disparity maps.
 - Applies a fully vectorised left-right consistency check and sets inconsistent pixels to zero.
- **fill_holes():**
 - Applies the three-stage hole-filling pipeline: morphological closing, Navier-Stokes inpainting, and guided or bilateral filter smoothing.
 - Reports the original valid pixel coverage percentage before filling.
- **to_metric_depth():**
 - Converts the disparity map to a depth map in centimetres (or normalised units) using the stereo depth formula.
 - Clips values at the 1st and 99th percentiles to remove outliers.
- **run_pipeline()**

- The main orchestrator function. It calls each stage in sequence, handles image size mismatches by resizing to the smaller dimension, and returns a dictionary containing all intermediate and final results.
- **display_results()**
 - Handles visualization using Matplotlib, displaying all eight panels in a 2×4 grid.
 - Embeds vital pipeline statistics (like the hole-fill percentage and metric depth ranges) directly into the subplot titles and colorbar labels for immediate visual feedback.
 - Creates side-by-side visual comparisons of the depth maps before and after mathematical inversion, allowing the user to view the geometry where either the background or the foreground is emphasized.

4. Results

- The system successfully produces dense depth maps for a wide variety of stereo image pairs, including standard benchmarks and handheld photographs taken with a smartphone.
- The StereoSGBM algorithm combined with the left-right consistency check significantly reduces the number of invalid pixels compared to simpler disparity methods, yielding cleaner and more complete depth maps.
- The multi-stage hole-filling pipeline recovers most remaining invalid regions, resulting in near-complete coverage even in texture-poor areas such as smooth walls or sky.
- When the baseline is provided, the metric depth output gives physically interpretable distances in centimetres, making the system usable for real-world scale estimation.
- The eight-panel figure and the detailed **console diagnostic log** allow users to verify each stage of the pipeline—from RANSAC inlier retention to disparity coverage—and quickly identify any geometric issues with a given image pair.



Task 8: High Dynamic Range (HDR) Imaging

1. Objective

- The goal of this task was to implement a custom, modular HDR pipeline from scratch to reconstruct a 32-bit high-dynamic-range radiance map from a sequence of multi-exposure LDR (Low Dynamic Range) images. The process involves precise physical light calculation,

image alignment, radiance merging using weighted exposure values, and global tone mapping to visualize the result on standard displays.

2. Methodology

➤ The custom pipeline consists of the following key stages:

- **Pre-processing & Exposure Validation:** Instead of relying on auto-sorting heuristics, the system parses the raw exposure values provided by the dataset. To ensure accurate physical light calculations, inverse shutter speed values (e.g., 1024) are mathematically converted into true exposure times in seconds (e.g., 1/1024) before processing.
- **Image Alignment (ECC):** Hand-held HDR photography often suffers from slight camera shakes. We utilized Enhanced Correlation Coefficient (ECC) maximization via `cv2.findTransformECC`. The middle image in the sequence is designated as the stationary reference frame, and all other images are warped using a 2D Euclidean motion matrix (translation and rotation) to ensure pixel-perfect overlap.
- **Linearization (Inverse Gamma):** Standard images are encoded with a non-linear sRGB display curve. We applied an inverse gamma transformation ($I^{2.2}$) before merging to ensure all radiance calculations were performed in a true, linear light space.
- **Radiance Merging:** The unbounded HDR radiance map is reconstructed using a weighted average approach:

$$Radiance = \frac{\sum w(I) \cdot (I/t)}{\sum w(I)}$$

where I is the linear pixel intensity, t_i is the true exposure time in seconds, and w_i is a weight function that prioritizes well-exposed midtones over saturated highlights or noisy, under-exposed shadows.

- **Tone Mapping (Reinhard):** To convert the infinite-range radiance map back to a viewable 8-bit image, we implemented a custom Global Reinhard Operator. First, the log-average luminance of the scene is calculated and scaled by a customizable key parameter (set to 0.10) to control the baseline exposure. Then, the non-linear compression function is applied:

$$L(out) = \frac{L_{in}}{1 + L_{in}}$$

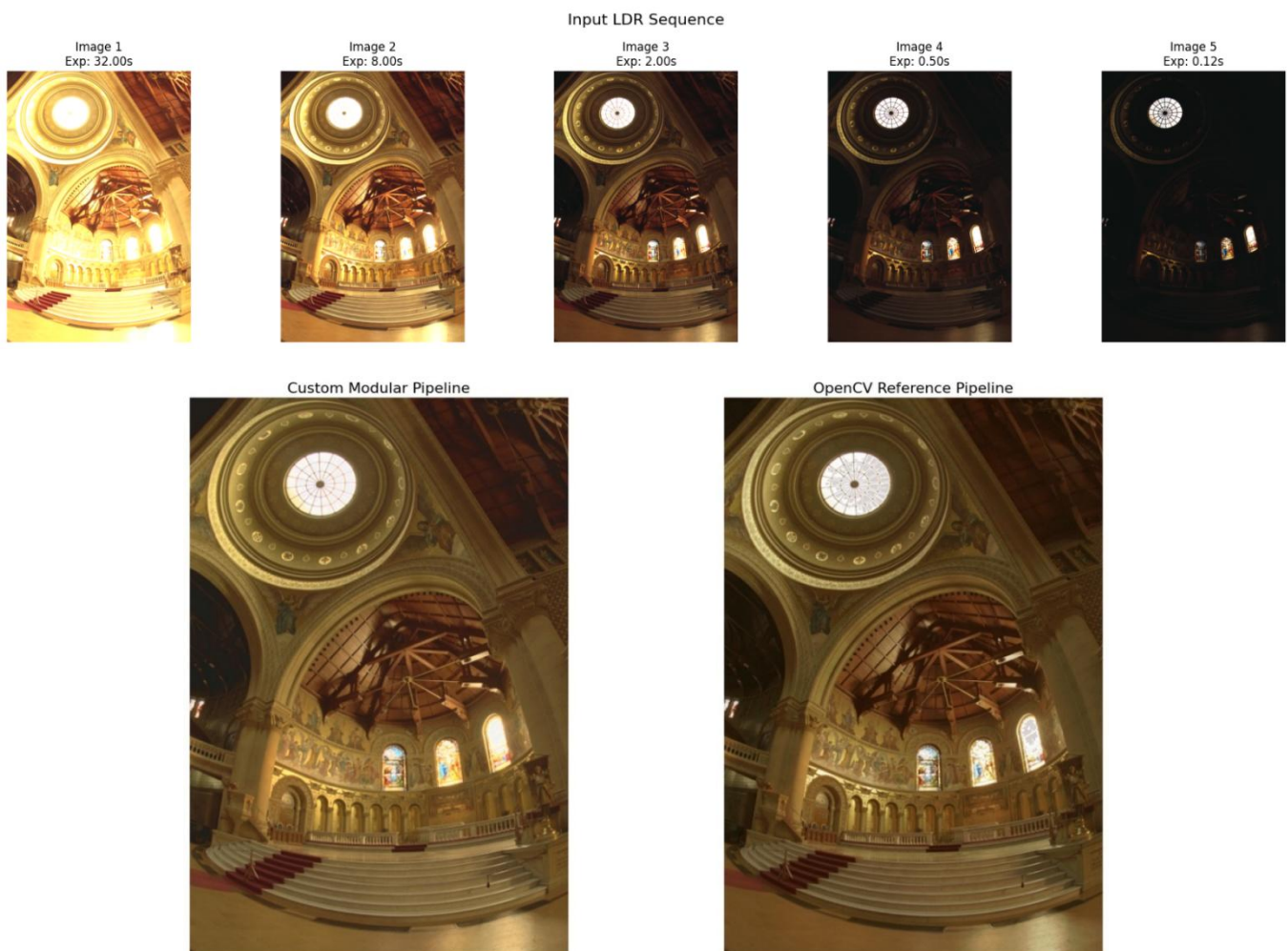
This safely compresses the highlights while preserving local contrast, followed by a standard $\frac{1}{2.2}$ gamma correction for display encoding.

3. Implementation Details

- **Libraries:** The implementation utilizes OpenCV for image ingestion and alignment, and NumPy for highly optimized, vectorized matrix operations across the multi-channel float arrays. Matplotlib is used for visualization.
- **Weighting Function:** We used a simplistic but highly effective luminance-based "tent" weighting scheme. The weight peaks at 0.5 luminance and drops to zero at the extremes (0.0 and 1.0) to maximize the Signal-to-Noise Ratio (SNR).
- **Robust Error Handling:** The ECC alignment function includes a try-except block. If optimization fails to converge (which is common when comparing extremely dark and extremely bright exposures where features are lost), the pipeline safely falls back to copying the original unaligned image, preventing crashes.

4. Results and Analysis

- **Detail Recovery:** The final custom output demonstrates a significant increase in visible detail compared to any single exposure in the input sequence.
- **Dynamic Range Preservation:** Details in the dark architectural elements are successfully recovered from the longest exposures (e.g., 32 seconds), while the vibrant details of the stained glass windows—which are entirely "blown out" in most frames—are perfectly preserved from the shortest fraction-of-a-second exposures.
- **Algorithm Validation:** The custom modular output was plotted side-by-side against a reference HDR image generated using OpenCV's highly optimized, built-in toolset (Debevec Merge OpenCV Reinhard). The visual comparison successfully verified the mathematical accuracy and capability of the custom pipeline.



Conclusion

This project successfully demonstrates the design and implementation of a comprehensive suite of advanced computer vision and computational photography pipelines. By building these systems from the ground up, the project highlights the enduring power and efficiency of classical, mathematically grounded image processing techniques to solve complex real-world visual problems without relying on opaque deep learning models.